

PreDAC: An Efficient Framework of Pre-Refining Enhanced Design Space Exploration for Approximate Computing

Ziying Cui*, Ke Chen*, Bi Wu, Yu Gong, Chenggang Yan, and Weiqiang Liu
Nanjing University of Aeronautics and Astronautics, Nanjing, China
{chen.ke, liuweiqiang}@nuaa.edu.com

Abstract—Approximate computing has emerged as a promising solution in energy-efficiency applications. Recently, attention has shifted from approximate components to Design Space Exploration (DSE) algorithms. However, traditional DSE algorithms face challenges in efficiently obtaining optimal solutions within large and complex design spaces. This paper introduces a pre-refining enhanced design space exploration framework that provides customized design space and cost-performance formula for applications. Experimental results demonstrate that integrating this pre-refining step into various DSE algorithms leads to substantial performance gains, including up to 87× speedup and a 23% improvement in hardware overhead. Moreover, the innovative cost-performance-based DSE algorithm attains a 7.7× acceleration and further optimizes hardware metrics by an additional 8.8% compared to advanced frameworks employing the same pre-refinement.

Index Terms—Approximate Computing, Design Space Exploration, High-Level Synthesis

I. INTRODUCTION

Approximate computing has emerged as an effective paradigm to mitigate hardware overheads by leveraging the inherent fault tolerance of applications in the post-Moore era. This approach significantly enhances the design methodologies for artificial intelligence (AI), the Internet of Things (IoT), and communication systems. Despite the existence of numerous approximate design methods [1]–[3], the process of crafting an approximate scheme tailored to a specific application remains a time-consuming and labor-intensive task. Currently, the derivation of such approximation schemes predominantly relies on manual expertise, with limited support from Electronic Design Automation (EDA) tools. To address these challenges, researchers have increasingly turned their attention to the development of design space exploration (DSE) algorithms. Existing DSE approaches can be broadly classified into two categories. The first employs heuristic algorithms [4]–[7], which iteratively search within the design space but often require substantial runtime. The second category adopts stepwise approximation methods, guided by metrics such as cost-performance trade-offs [8] or impact factors [9]. These methods achieve reduced synthesis and validation times but

often produce suboptimal results. Consequently, the algorithms proposed in existing studies face significant limitations: excessive runtime, poor scalability, or unsatisfactory outcomes due to the vastness of the design space. Moreover, existing approximate libraries are constrained in scope and fail to comprehensively capture the full design space.

This paper proposes PreDAC, an efficient pre-refining enhanced design space exploration framework. PreDAC considers application-specific input distributions, optimization objectives, and accuracy constraints to construct a comprehensive, redundancy-free design space tailored to the application. Additionally, an optimized DSE algorithm is introduced, which employs a stepwise approach, progressing from the preliminary exact design towards the periphery of the design space—where designs with higher error and minimal hardware overhead dominate—guided by a cost-performance ratio rather than random generation. Furthermore, a novel strategy is incorporated to further minimize hardware overhead. PreDAC is evaluated across the design spaces of four widely used applications, demonstrating favorable results. The key contributions of this work are summarized as follows:

- A generic approximation library is constructed by integrating commonly used approximate multipliers.
- A novel two-stage pre-refining step is introduced, leveraging application characteristics to provide a customized design space compatible with arbitrary DSE algorithms.
- An enhanced DSE algorithm is proposed, incorporating a cost-performance formula with adjustable parameters and a fine-tuning strategy.
- PreDAC achieves an average speedup of 7.7× compared to FPAX [8] and delivers an additional 8.8% optimization over JS [9].

II. PROPOSED PRE-REFINING FRAMEWORK

The proposed PreDAC framework consists of two main steps, as illustrated in Fig. 1. The first step involves the pre-refinement of the approximate multiplier library, which serves as a preparatory phase and can be seamlessly integrated into any design space exploration algorithm. The second step entails an enhanced design space exploration algorithm, which systematically transitions from the initial fully accurate design to configurations with progressively higher error and reduced hardware overhead, guided by the cost-performance ratio.

*These authors contributed to the work equally and should be regarded as co-first authors.

This work is supported by grants from the National Key R&D Programme of China (2022YFB4500200), National Nature Science Foundation of China (62425404, 92364201).

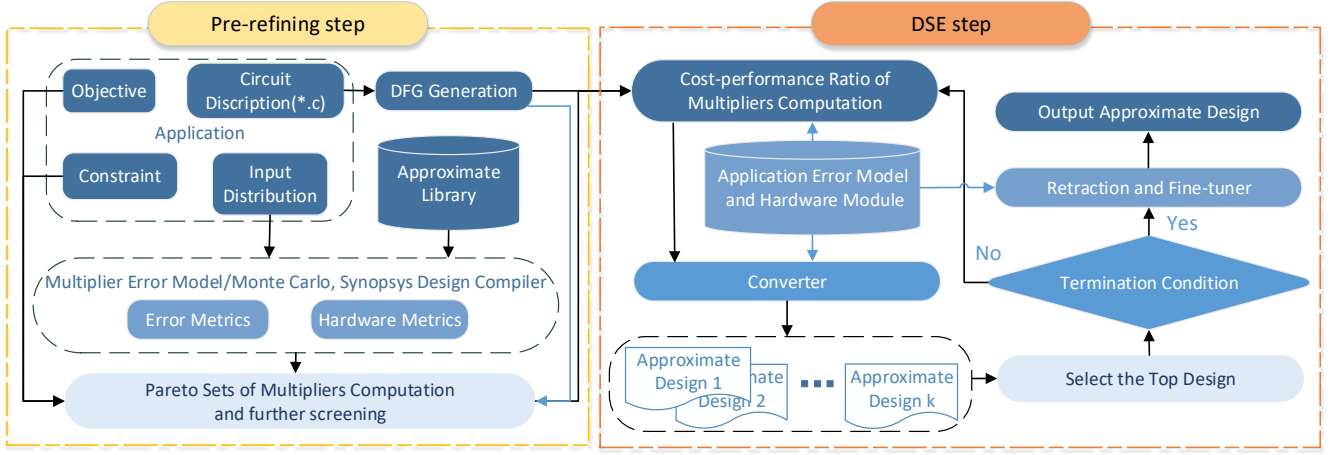


Fig. 1. The frame diagram of PreDAC.

A. Approximate Multipliers Library

Since multiplication is one of the most frequently used arithmetic operations in a wide range of algorithms, approximate multipliers are employed to construct a comprehensive approximation library capable of accommodating diverse application requirements. Approximate multipliers can be broadly classified into two categories: manually designed multipliers and automatically generated multipliers. To ensure comprehensive coverage, this paper selects representative samples from both categories. In the category of manually designed multipliers, the widely used DRUM multipliers [10] and ETM multipliers [11] are included, resulting in a total of 10 approximate multipliers, achieved by tuning configurable parameters. Additionally, a commonly employed approximate multiplier design methodology [12], which incorporates techniques such as radix-4 Booth encoding and decoding, input truncation and compensation, partial product truncation and compensation, and approximate compressors, is also considered. This methodology yields 34 approximate multipliers by varying the relevant parameters. In the category of automatically generated multipliers, 11 approximate multipliers derived using a genetic algorithm [13] are selected for inclusion in the library. In total, the approximation library comprises 55 approximate multipliers, numbered sequentially from 1 to 55, providing a robust and diverse foundation for exploring and optimizing hardware designs across various applications.

B. Pre-Refining of the Approximate Library

The size of the design space grows exponentially with the number of multipliers in the library. For example, in a 13-tap FIR filter, if the number of available approximate multipliers is n , the design space expands to 13^{n+1} . By reducing n from 8 to 7, approximately 10 billion design configurations can be eliminated, highlighting the necessity of refining the approximate library prior to executing the DSE algorithm. To address this, the 55 approximate multipliers in the library undergo a pre-refinement step, which identifies optimal approximate

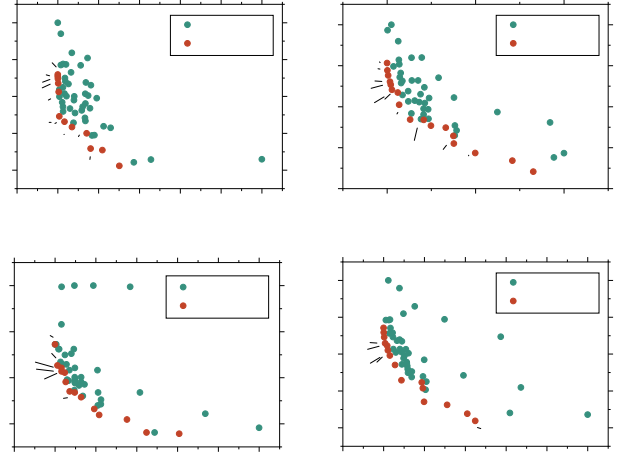


Fig. 2. Pareto set of multipliers for four distinct applications along with corresponding hardware and accuracy metrics.

configurations tailored to specific applications. This process ensures that the design space remains both manageable and relevant, significantly enhancing the efficiency of subsequent design space exploration and optimization.

In addition, different applications exhibit unique input distributions, and the various error metrics associated with multipliers have distinct strengths and weaknesses. As a result, the error characteristics of a given multiplier can vary significantly depending on the input distribution and the selected error metric. Furthermore, the behavior of a multiplier may change dramatically depending on the ultimate optimization objective of the application, such as minimizing area, power consumption, or delay. To illustrate these considerations, four distinct applications, RGB2YCbCr, 13-tap FIR filter, Gaussian Blur filter, and 3x3 convolution were selected, each paired with corresponding hardware and accuracy metrics. The multiplier

error model described in [12] was employed to compute error metrics, as it offers a more time-efficient alternative to the Monte Carlo method. Hardware metrics were synthesized using Synopsys Design Compiler (DC) to ensure accuracy. The resulting Pareto-optimal sets of multipliers for these applications are highlighted in red in Fig. 2. The corresponding hardware metrics are area, power consumption, latency, and a comprehensive hardware metric. The accuracy metrics are MED (Mean Error Distance), MSE (Mean Square Error Distance), MAX_ED (Maximum Error Distance), and a comprehensive accuracy metric. As shown in Fig. 2, the Pareto sets of multipliers differ significantly across applications, emphasizing the need for application-specific refinements. Since even a single reduction in the number of approximate multipliers can drastically impact the design space, a second screening of the approximate library is necessary. This screening considers the error constraints specific to each application, complementing the initial pre-refinement step.

The proposed pre-refining step is structured into two distinct parts. The first part involves selecting the Pareto-optimal set of multipliers from the library, guided by the application's specific input distribution. The second part focuses on eliminating multipliers that exhibit excessive errors, ensuring compliance with the given error constraints. The detailed procedure is outlined in Algorithm 1.

Algorithm 1 Pre-refining

Input: DFG (G); Input Distribution (D); 55 Approximate Multipliers (M_o); Objective (O); Error Constraints (EC);
Output: Multipliers for DSE (M_l); Abandon for Level (AL);

```

1: Initialize:  $HW_{metric} = \emptyset, Err_{metric} = \emptyset, AL = \emptyset$ ;
2: for  $i \leftarrow 1$  to 55 do
3:    $HW = DC(D, O, M_o[i])$ ;
4:    $HW_{metric}.append(HW)$ ;
5:    $Err = Multiplier\_Error\_model(D, O, M_o[i])$ ;
6:    $Err_{metric}.append(Err)$ ;
7: end for
8:  $M_l = Pareto\_set(HW_{metric}, Err_{metric})$ ;
9:  $(N_c, Mul_c) = Classifier(G)$ ;
10: for  $i \leftarrow 1$  to  $N_c$  do
11:    $Mul = Random\_select(Mul_c[i])$ ;
12:   for  $j \leftarrow length(Mul_{pareto})$  to 1 do
13:      $abandon = \emptyset$ ;
14:      $Mul = Mul_{pareto}[j]$ ;
15:      $Err = Application\_Error\_model(D, G, O, Mul)$ ;
16:     if  $Err \leq EC$  then
17:       break
18:     else
19:        $abandon.append(j)$ ;
20:     end if
21:   end for
22:    $AL.append(abandon)$ ;
23: end for
24: return  $M_l, AL$ 

```

The first section of Algorithm 1 (steps 2–8) utilizes input

distributions, objective metrics, and the 55 approximate multipliers to compute the Pareto-optimal set, following the computation of the corresponding hardware and error metrics. The input dataset is transformed into a probability density function, which is integrated into the error model of the approximate multipliers. This enables the cumulative error generated by each input case to be weighted by its probability, effectively incorporating the input distribution into the analysis. The resulting Pareto sets of multipliers are arranged in ascending order of error magnitude, facilitating subsequent operations.

The second component of Algorithm 1 (steps 9–23) is then executed. The input to the framework is a high-level language model (*.c) of the application, which is first converted into a Data Flow Graph (DFG) using GAUT [14]. Candidate multipliers are classified by their levels in the DFG, determined by the number of operations performed from the input to the multiplier, using the *Classifier* function in Algorithm 1. This approach assumes that multipliers at the same level in DFG have comparable impacts on the final result and, thus, should be categorized together. Next, for each class, a candidate multiplier is randomly selected for replacement with the multiplier exhibiting the largest error in the library (steps 11 and 14). The application error model [15] is then used to compute the final result and verify whether it satisfies the prescribed error constraints. If the error exceeds the threshold, the approximate multiplier is deemed unsuitable, and the replacement strategy is excluded from DSE algorithms for that category (step 19). In the subsequent step, the multiplier with the second-highest error is selected and replaced, and the process is repeated until the error meets the specified limits or all library multipliers have been evaluated. Otherwise, the operation is repeated for the subsequent class of multipliers. This process is iteratively applied to all classes of multipliers, ensuring that only one candidate multiplier is replaced per iteration, while others remain in exact form.

The multipliers obtained from the pre-refining stage are sorted and numbered based on their error magnitudes in ascending order. These refined multipliers, along with the abandon set, constitute the approximation library for the DSE algorithm. This pre-refinement step significantly accelerates the exploration process by reducing the design space and simultaneously delivers superior results compared to employing the full unrefined multiplier library.

C. Cost-Performance ratio Based Design Space Exploration Method

In this paper, the proposed DSE algorithm transitions systematically from an initial error-free design to designs with progressively higher error and reduced hardware overhead, guided by the cost-performance ratio. The algorithm consists of four principal stages. The initial step is Cost-Performance Calculation, which compute the cost-performance ratio for the multiplier replacement schemes, serving as the primary evaluation metric. The second step is Approximate Design Generation, which fed the computed cost-performance ratio into the converter to obtain a variety of approximate designs.

The third step is Validation Against Abort Condition, which determine whether the generated design meet the predefined abort conditions. If the conditions are not satisfied, the process loops back to the first step. The fourth step is Retraction and Fine-Tuning, which enable the algorithm proceeds to a retraction and fine-tuning phase, optimizing the final design further before outputting the result if the abort conditions are met. Throughout the algorithm, error metrics and hardware metrics are obtained using an applied error model [15] and a hardware model [16], ensuring accurate evaluation of both approximation quality and hardware performance.

1) *Application Error Model and Hardware Model:* During the process of DSE, it is crucial to perform regular verification to ensure that the current design adheres to specified error constraints while optimizing hardware utilization compared to an exact counterpart. In practice, Monte Carlo methods and DC synthesis are the most frequently employed techniques for achieving these objectives. However, Monte Carlo methods are computationally intensive. Although reducing the number of iterations can decrease runtime, it may lead to intolerable errors and unsatisfactory outcomes in DSE. This issue arises due to the unclear relationship between the number of iterations and the resulting accuracy. From a hardware perspective, increasing the frequency of interfacing with DC during DSE exacerbates system complexity and significantly extends runtime. To address these challenges, this paper employs the analytical model proposed in [15], which computes final errors with higher accuracy and is three orders of magnitude faster than Monte Carlo methods. For hardware evaluation, the theoretical analysis method introduced in [16] is used as a substitute for DC synthesis, aiming to reduce DSE complexity. It is important to note that all outputs are validated using DC to ensure the accurate assessment of approximate designs.

2) *Cost-Performance Computation:* In PreDAC, the approximation process involves sequentially refining the candidate multipliers within an application. To systematically guide this refinement, it is necessary to define the depth—the number of candidate multipliers currently under consideration. This facilitates the incremental approximation of multipliers in a structured manner. For each iteration, the cost-performance ratio is calculated to evaluate the trade-offs of replacing a candidate multiplier, starting from the current depth to the final multiplier, with a substitute from the approximate library that exhibits a higher error. This cost-performance ratio serves as the foundation for executing the replacement strategy. It is assumed that the replacement cost-performance ratios for multipliers of the same class are identical.

To illustrate, consider a 3x3 convolution operation with three multipliers in the approximation library, labeled 1, 2, and 3. Initially, all candidate multipliers are exact (denoted as 0). Following several iterations, the resulting configuration might appear as [1, 1, 0, 1, 0, 2, 1, 1, 0]—a total of nine candidate multipliers of the same class. At this stage, the cost-performance ratio must be computed for various replacement scenarios. For example, the fourth multiplier could be replaced

with approximate multipliers 1, 2, or 3; similarly, the fifth multiplier might be replaced with multiplier 3, and the sixth multiplier with multipliers 2 or 3. This paper introduces a novel cost-performance formula with adjustable hyperparameters, enabling its calibration to meet specific accuracy and hardware requirements. The formula is defined as follows:

$$\begin{aligned} a &= \frac{HO_o^x - HO_l^x}{HO_o^x - HO_{min}^x} \\ b &= \frac{error_l^y - error_o^y}{error_{max}^y - error_o^y} \\ c_p &= \left| \frac{a}{b} \right|, \end{aligned} \quad (1)$$

where HO_o represents the hardware overhead of the original design, HO_l is the hardware overhead after replacement, and HO_{min} denotes the hardware overhead of replacing the multiplier with the largest error. Similarly, $error_l$ refers to the error of the replaced design, while $error_o$ denotes the error of the original design, and $error_{max}$ corresponds to the error of the largest-error design. Notably, the designs represented by HO_{min} and $error_{max}$ are identical. Both a and b are all normalized, while c_p represents the cost-performance ratio.

Consider two scenarios where the error metrics are MSE and MED, respectively, while the hardware metrics remain constant. Due to the significant numerical differences between MSE and MED, an unmodified cost-performance formula may disproportionately favor multipliers with lower MSE, leading to slower convergence. Conversely, for MED, the formula may overly prioritize lower hardware overhead, potentially overlooking superior designs and yielding suboptimal outcomes. To address this, distinct hyperparameter settings are applied for different error metrics to ensure balanced cost-performance values. For instance, when the hardware metrics focus on area or power consumption and the error metrics use MSE, the hyperparameters $x = 3, y = 1/2$ are recommended. In contrast, for MED, selecting $x = 3, y = 1$ provides better results. Additionally, error metrics expressed in terms of Signal-to-Noise Ratio (SNR) are converted to MSE for consistency during DSE.

3) *Approximate Design Generation:* In this approach, the FPAX [8] converter is utilized to facilitate the generation of approximate designs. Initially, two distinct designs are created by applying the two most cost-effective replacements, prioritizing them in alternating order. These two designs are then subjected to randomization processes inspired by genetic algorithms, incorporating crossover and mutation principles to produce additional designs. The cost-performance ratio for each design is computed using the formula presented in (1). The design with the highest cost-performance ratio is selected as the result of the current iteration.

4) *Validation Against Abort Condition:* The resulting design generated by II-C3 undergoes evaluation to determine whether its error margin complies with the predefined constraints. If the error margin violates these constraints, the termination condition is triggered, and the retraction and fine-tuning phase is initiated to refine the design. Conversely, if the

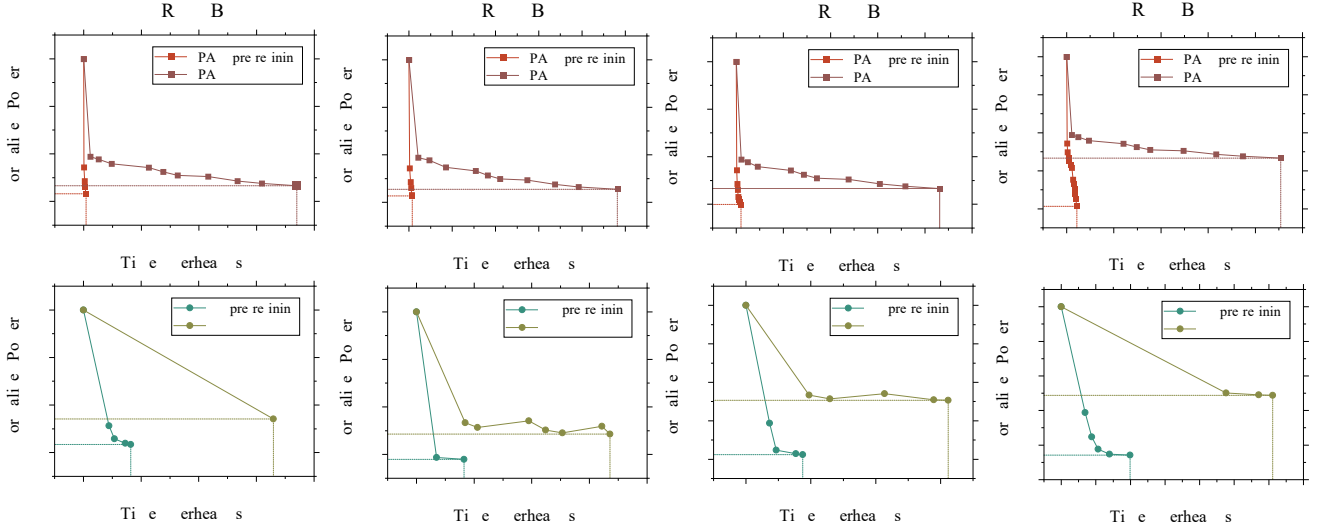


Fig. 3. Comparison between the DSE process with and without a pre-refining step under four different SNR constraints for power consumption optimization of 13-tap FIR, using FPAX and JS framework, respectively.

design satisfies the error constraints, the process continues to the next iteration.

5) *Retraction and Fine-Tuning*: When the error exceeds the prescribed constraint, a retraction step is applied to revert the candidate multiplier to the state from the preceding iteration. This process begins at the current depth and proceeds forward until the constraint is satisfied. Due to the inherent imprecision of retraction and the potential for further approximation in the resulting output, a subsequent fine-tuning step is introduced to optimize the design.

During fine-tuning, each candidate multiplier is incrementally approximated by one level, replacing it with the next higher-numbered approximate multiplier from the library. These replacements are performed individually for each candidate to generate multiple designs. Designs that satisfy the error constraint are then selected for evaluating their cost-performance ratios. Replacements are executed in descending order of cost-performance ratio until the error constraint is no longer met. At this point, the prior step is restored, and the corresponding design is finalized as the output. If all replacement strategies are applied without breaching the error constraint, the process undergoes a single additional iteration to further refine the design based on the revised configuration.

III. EVALUATION AND ANALYSIS

To evaluate the effectiveness of the PreDAC framework, four widely used applications were selected for analysis: RGB2YCbCr, 3x3 Convolution, 13-tap FIR Filter, and Gaussian Blur Filter. These applications, as depicted in Fig. 2, are representative of image processing, artificial intelligence, and communication systems. While PreDAC can accommodate various error and hardware metrics, this study focuses on approximate computation gains in terms of area and power consumption. Accordingly, these two hardware metrics were chosen for evaluation.

For accuracy metrics, the MED and SNR were selected. MED evaluates whether the approximation error is positively or negatively biased, whereas SNR serves as a key performance metric in a broad range of applications, including image processing and communication systems. The comparative frameworks include JUMP search (JS) [9] and FPAX [8]. Both frameworks utilize a stepwise approximation approach instead of randomly generating designs within the design space, ensuring faster runtime compared to other methods. The evaluation process in this paper is divided into two main sections: (1) assessing the pre-refining step and (2) evaluating the enhanced DSE algorithm.

The PreDAC framework was implemented in Python, and experiments were conducted on a system with the following specifications: Intel(R) Xeon(R) Gold 6258R CPU (2.70 GHz) with 112 cores and 192 GB of RAM.

A. Pre-refining Step Evaluation

To evaluate the effectiveness of the proposed pre-refining step, experiments were conducted using the JS [9] and FPAX [8] frameworks, with and without the pre-refining step, respectively. The objective of hardware optimization was to minimize power consumption, and the design space exploration for the 13-tap FIR filter was performed under four SNR constraints: 28 dB, 25 dB, 22 dB, and 19 dB. The results are depicted in Fig. 3, where the horizontal axis represents runtime and the vertical axis represents power consumption, normalized to the exact design.

In the FPAX framework, the number of iterations is limited to 10 due to the excessive runtime when the pre-refining step is omitted. The FPAX algorithm computes the cost-performance ratio for each multiplier in the approximate library during every iteration. Notably, the number of multipliers before pre-refining is approximately five times larger than after pre-

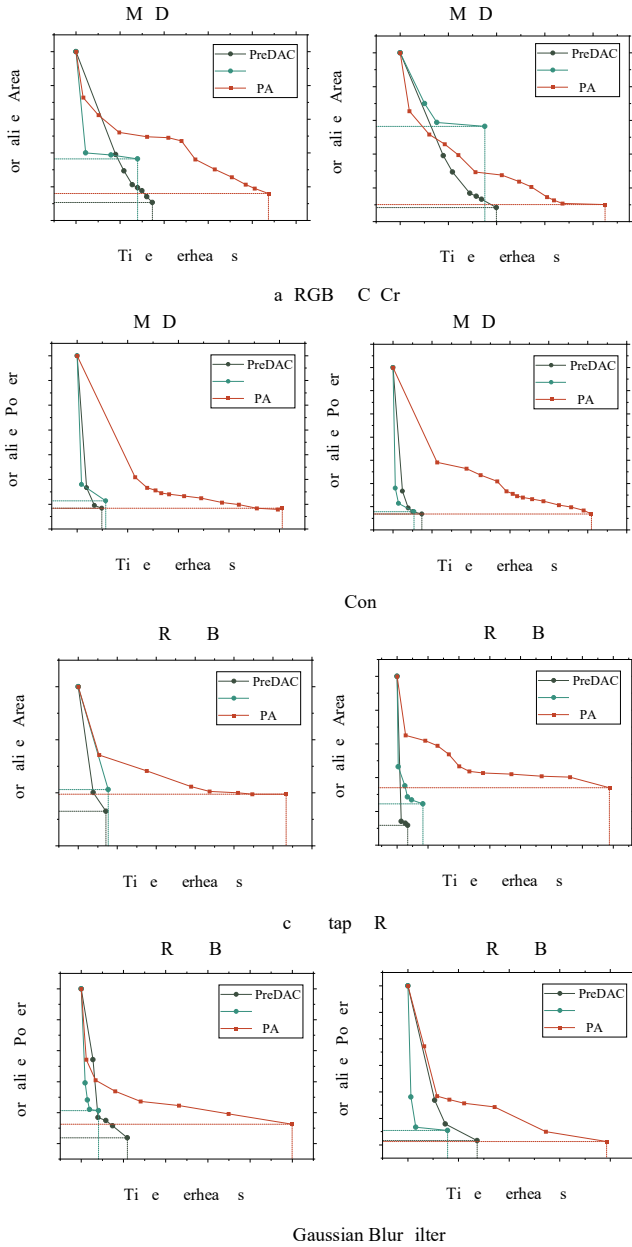


Fig. 4. Comparison of Exploration Efficiency between PreDAC, JS and FPAX using the same Pre-refining Step.

refining, leading to a speedup of up to 87 $\times$ when the pre-refining step is applied, as shown in Fig. 3.

A similar runtime issue exists in the JS algorithm, although its error metric is calculated only once per iteration. As a result, the speedup for JS is approximately 4 $\times$ on average when the pre-refining step is employed. Without pre-refining, the approximate library contains numerous non-Pareto multipliers with higher errors and worse power consumption, unnecessarily inflating the design space, slowing convergence, and yielding suboptimal outcomes. By implementing the pre-refining step, the JS algorithm achieves a further 23% reduction in power consumption compared to the scenario without pre-

refinement. This demonstrates the pre-refining step's ability to significantly enhance both efficiency and optimization quality in the design space exploration process.

B. DSE Algorithm Evaluation

The proposed enhanced DSE algorithm was compared against the JS and FPAX frameworks under identical pre-refining conditions. The experiment incorporated two error metrics, two hardware metrics, and four fault-tolerant applications. The experimental results are shown in Fig. 4. The selected MED constraints are 1000 and 1400 in order, and the SNR are 25dB and 19dB.

As illustrated in Fig. 4, the proposed algorithm demonstrated comparable or superior performance relative to FPAX, achieving an average speedup of 7.7 $\times$. This improvement is attributed to the use of an application error model instead of Monte Carlo simulations, which significantly reduces the computational time required for error metric evaluation—a major component of the DSE algorithm runtime. Additionally, the enhanced cost-performance formula introduced in this study can be adaptively calibrated to balance runtime and design quality. This adaptability directs iteration outcomes towards maximizing gains and facilitates early termination, further minimizing runtime.

Compared to the JS algorithm, PreDAC achieved a similar runtime in most cases, with any runtime increases falling within acceptable limits. Furthermore, the enhanced algorithm delivered an average improvement of 8.8% in hardware overhead optimization. The precision of the application error model used in this study surpasses that of the Monte Carlo method, while the enhanced cost-performance formula enables the selection of appropriate hyperparameters to guide the DSE algorithm effectively. Finally, the retraction and fine-tuning steps proposed here exploit the remaining approximation space to optimize hardware overhead further while adhering to the specified error constraints.

IV. CONCLUSION

The PreDAC framework is proposed to streamline design space exploration by providing a library of customized approximation multipliers tailored to the application's input distribution, optimization objectives, and accuracy constraints. The framework incorporates an enhanced cost-performance formula and a fine-tuning step. This integration allows for an optimal balance between performance and design quality. The effectiveness of the pre-refining method and the improved DSE algorithm was evaluated independently. When combined with the pre-refining step, FPAX achieved the highest speedup, up to 87 $\times$ compared to its original implementation. Meanwhile, the JS algorithm [9] demonstrated a 23% improvement in hardware overhead optimization. Under the condition of using the same pre-refining steps, the PreDAC framework achieved an additional average optimization of 8.8% in design quality while maintaining a runtime comparable to that of JS. Moreover, PreDAC delivered results that were comparable to or better than FPAX, achieving an average speedup of 7.7 $\times$.

REFERENCES

- [1] R. Zendegani, M. Kamal, M. Bahadori, A. Afzali-Kusha, and M. Pedram, "Roba multiplier: A rounding-based approximate multiplier for high-speed yet energy-efficient digital signal processing," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 2, pp. 393–401, 2017.
- [2] H. Jiang, L. Liu, P. P. Jonker, D. G. Elliott, F. Lombardi, and J. Han, "A high-performance and energy-efficient fir adaptive filter using approximate distributed arithmetic circuits," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 1, pp. 313–326, 2019.
- [3] M. Shafique, R. Hafiz, S. Rehman, W. El-Harouni, and J. Henkel, "Invited: Cross-layer approximate computing: From logic to architectures," in *2016 53rd ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2016, pp. 1–6.
- [4] V. Mrazek, M. A. Hanif, Z. Vasicek, L. Sekanina, and M. Shafique, "autoax: An automatic design space exploration and circuit building methodology utilizing libraries of approximate components," in *2019 56th ACM/IEEE Design Automation Conference (DAC)*, 2019, pp. 1–6.
- [5] M. Vaeztourshizi, M. Kamal, and M. Pedram, "Egan: A framework for exploring the accuracy vs. energy efficiency trade-off in hardware implementation of error resilient applications," in *2020 21st International Symposium on Quality Electronic Design (ISQED)*, 2020, pp. 438–443.
- [6] Z. Zhang, T. Chen, J. Huang, and M. Zhang, "A fast parameter tuning framework via transfer learning and multi-objective bayesian optimization," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. Association for Computing Machinery, 2022, p. 133–138. [Online]. Available: <https://doi.org/10.1145/3489517.3530430>
- [7] L. Ferretti, J. Kwon, G. Ansaloni, G. D. Guglielmo, L. P. Carloni, and L. Pozzi, "Leveraging prior knowledge for effective design-space exploration in high-level synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 11, pp. 3736–3747, 2020.
- [8] Y. Dou, C. Wang, H. Waris, R. Woods, and W. Liu, "Fpax: A fast prior knowledge-based framework for dse in approximate configurations," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 6, pp. 1650–1662, 2024.
- [9] L. Witschen, H. Ghasemzadeh Mohammadi, M. Artmann, and M. Platzner, "Jump search: A fast technique for the synthesis of approximate circuits," in *Proceedings of the 2019 Great Lakes Symposium on VLSI*, ser. GLSVLSI '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 153–158.
- [10] S. Hashemi, R. I. Bahar, and S. Reda, "Drum: A dynamic range unbiased multiplier for approximate applications," in *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2015, pp. 418–425.
- [11] K. Y. Kyaw, W. L. Goh, and K. S. Yeo, "Low-power high-speed multiplier for error-tolerant application," in *2010 IEEE International Conference of Electron Devices and Solid-State Circuits (EDSSC)*, 2010, pp. 1–4.
- [12] Z. Cui, K. Chen, B. Wu, C. Yan, Y. Gong, and W. Liu, "A time efficient comprehensive model of approximate multipliers for design space exploration," in *2024 IEEE 31st Symposium on Computer Arithmetic (ARITH)*, 2024, pp. 116–123.
- [13] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, "Evoapprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2017, 2017, pp. 258–261.
- [14] E. Martin, O. Sentieys, H. Dubois, and J. Philippe, "Gaut: An architectural synthesis tool for dedicated signal processors," in *Proceedings of EURO-DAC 93 and EURO-VHDL 93- European Design Automation Conference*, 1993, pp. 14–19.
- [15] S. Chen, K. Chen, Z. Cui, and W. Liu, "A high-accuracy time-efficient error metric model for approximate computing circuits," in *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2024, pp. 94–99.
- [16] J. Castro-Godínez, J. Mateus-Vargas, M. Shafique, and J. Henkel, "Axhls: Design space exploration and high-level synthesis of approximate accelerators using approximate functional units and analytical models," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–9.